Real-Time Surface-Pressure Buoyancy with Adaptive Curved Clipping

Felix Stenberg
felixste@kth.se
KTH Royal Institute of Technology | DH2323 Computer Graphics and Interaction
crispigt.com/dh2323 | GitHub Repo

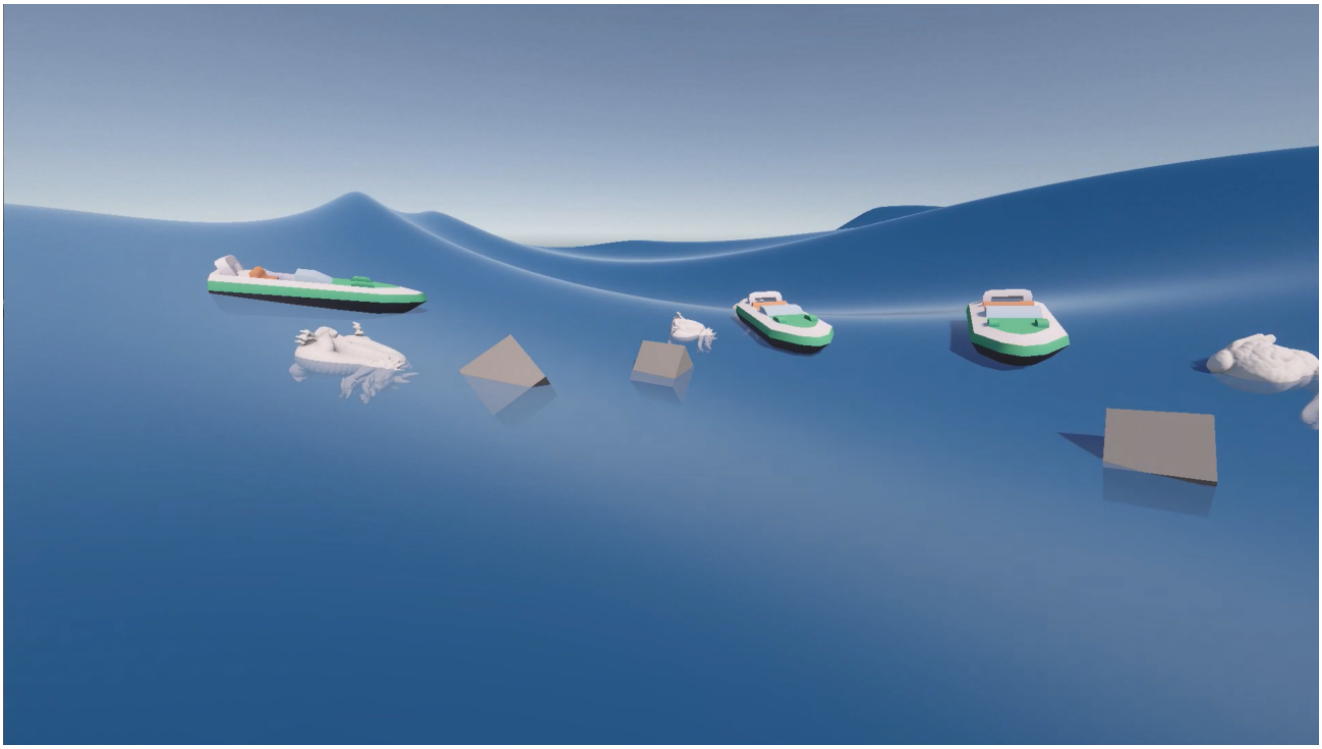

Figure 1: Floating objects in the wave scene.

Abstract

Real-time buoyancy in game engines typically relies on simplified point-sampling or volume-based methods that produce inaccurate torques on coarse meshes. Hirae et al. (2025) introduced a closed-form surface-pressure integrator that eliminates quadrature error, but assumes a locally flat waterline across each triangle, an approximation that breaks down when triangle edges are comparable in size to the wavelength. This project implements Hirae's integrator as a C++ native plugin for Unity, connected via P/Invoke to a Burst-compiled C# wave sampler, and introduces *Adaptive Curved Clipping*, a refinement scheme that samples the true Gerstner wave height along the clipping chord to correct the depth integral without geometric surface intersection. Evaluation against analytical solutions confirms machine-precision force accuracy and reproduces the 26.565° equilibrium tilt used by Hirae from Igarashi & Nakamura's (2007) long-rectangular-bar analysis. A convergence study against a 30,000-triangle ground truth shows that Adaptive Clipping with N=4 samples is fully converged, but offers negligible accuracy improvement over standard linear clipping, the dominant error source is mesh volume approximation, not waterline chord error. Performance stress testing with 100 simultaneous rigid bodies shows the linear C++ path sustains 363 FPS average, while the adaptive C# path degrades to single-digit FPS due to managed-code overhead.

Keywords: Buoyancy, hydrostatic pressure integration, mesh clipping, Unity, native plugin.

1. Introduction

1.1 Motivation

In a concurrent Game Design course, my team and I needed realistic buoyancy for a boat-based game. Unity's built-in physics offers only a flat buoyancy plane with no wave interaction, producing unconvincing behavior for any object more complex than a sphere. This motivated me to search for physically accurate, real-time buoyancy on dynamic wave surfaces.

The literature presents a clear progression. Fábián (2025) computes buoyancy for meshes intersecting a static flat water plane using exact volume- and surface-based constructions. Those methods are well suited to calm water, extending them to curved wave surfaces would require additional handling of non-planar waterline geometry. Hori (2021) showed, through surface integration of hydrostatic pressure, that the center of pressure coincides with the classical center of buoyancy. This supports the surface-pressure view of buoyancy;

Hirae et al. (2025) turn that view into a closed-form per-triangle integrator that computes exact force and torque without quadrature error, enabling accurate simulation even on coarse meshes.

1.2 Research Question

Hirae's integrator clips partially submerged triangles against a straight chord between the two edge-water intersections. On coarse meshes in moderate waves, this chord diverges from the true curved waterline. The main question I wanted to answer with this project was: *Can a piecewise-linear waterline refinement close the accuracy gap between coarse-mesh linear clipping and dense-mesh ground truth, at acceptable runtime cost?*

1.3 Contributions

1. A Unity-integrated C++ DLL implementation of Hirae's closed-form integrator with a wave-agnostic architecture, including identification and correction of a factor of 2 error in the paper's printed torque equations.
2. *Adaptive Curved Clipping*: a refinement algorithm that samples the true wave height along the clipping chord and corrects the depth integral through the existing closed-form equations.
3. An empirical three-way comparison (linear clipping, adaptive clipping, dense-mesh ground truth) evaluating both accuracy and real-time performance.

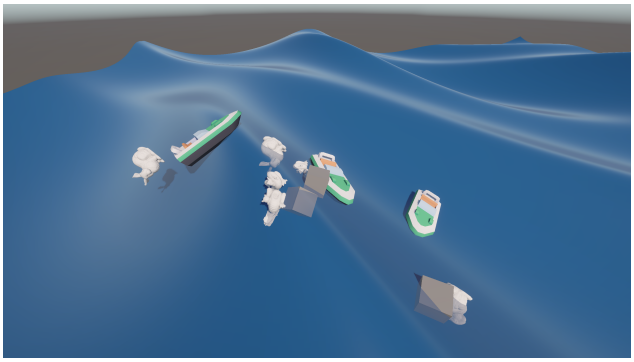

Figure 2: Same scene from another angle.

2. Previous Work

Fábián (2025) presents five buoyancy algorithms for meshes intersecting a flat water plane, including exact volume- and surface-based methods. The paper is explicitly framed around a resting planar fluid surface, in my setting, extending those constructions to curved waves would require additional handling of non-planar cap geometry. *Used for:* conceptual baseline for exact flat-water buoyancy.

Hori (2021) proved mathematically that the center of hydrostatic pressure obtained from surface integration coincides with the conventional center of buoyancy, first for a rectangular cross-section and then for arbitrary floating-body shapes using Gauss's integral theorem. *Used for:* theoretical justification of the surface-pressure view, buoyancy can be reasoned about from pressure over the wetted surface rather than only from displaced volume.

Hirae et al. (2025) derived closed-form expressions for per-triangle buoyancy force and torque from the surface-pressure integral. Their key insight is that centroid-based quadrature introduces "ghost torques" on coarse meshes because the torque integrand is not polynomial. The closed-form eliminates this error entirely. They also introduce angular momentum damping (scaling $\vec{L}$ rather than $\vec{\omega}$) to avoid axis-dependent artifacts. *Used for:* the core integrator equations (Eqs. 5–9), the angular momentum damping scheme, and the triangle clipping framework. During implementation, a typographical error was discovered in the printed equations (§5.1).

Flick (2018) provides the Catlike Coding tutorial *Waves: Moving Vertices*, including a MIT-licensed Gerstner wave implementation for Unity. *Used for:* the wave displacement function in both the C# physics code and the URP vertex shader, ensuring visual-physical synchronization.

Igarashi & Nakamura (2007) analyze the equilibrium orientation of a long rectangular bar in still water. Hirae use this as a validation target, and my equilibrium-tilt setup matches the same one-axis cross-sectional case by constraining a cube to rotate about only one axis. *Used for:* ground-truth comparison in the equilibrium-tilt test (§5.1).

3. Theory

3.1 Surface-Pressure Integration

For a rigid body partially submerged in fluid with density ρ, the net buoyancy force is the integral of hydrostatic pressure over the wetted surface Σ (Hori, 2021; Hirae et al., 2025):

$$\vec{F}_B = \iint_{\Sigma} \rho g\,(h-y)\,d\vec{S}$$

where h is the local water surface height, y is the depth coordinate, and $d\vec{S}$ is the outward-facing area element. By Gauss's divergence theorem, this surface integral equals the classical volume integral, recovering Archimedes' principle without requiring a closed surface or waterline cap.

3.2 Hirae's Closed-Form Integrator

For a triangle with vertices $\vec{v}_k = (x_k, y_k, z_k)$, $k = 1,2,3$ and per-vertex water heights h_k, Hirae derives exact expressions for force and torque. The force (Eq. 5) requires only the area vector $\vec{S} = \frac{1}{2}(\vec{v}_2 - \vec{v}_1) \times (\vec{v}_3 - \vec{v}_1)$:

$$\vec{F}_i = \frac{\rho g}{3}\left(-3\bar{h} + y_1 + y_2 + y_3\right)\vec{S}_i$$

where $\bar{h}$ is the average water height. No square root is needed, the area and normal are encoded together in $\vec{S}$.

The closed form is exact because the submerged triangle is integrated in barycentric coordinates. The depth term is linear over the triangle, so its area moments have fixed values:

$$\int_T \lambda_i \, dA = \frac{A}{3}, \quad \int_T \lambda_i^2 \, dA = \frac{A}{6}, \quad \int_T \lambda_i \lambda_j \, dA = \frac{A}{12} \, (i \neq j).$$

The force only needs the first moment of pressure. The torque needs the second moments because the lever arm is multiplied by the pressure field. This is why evaluating force at the centroid can look plausible while still producing incorrect torques on coarse meshes.

The torque about the center of mass $\vec{x}_{\text{CoM}}$ (Eq. 6) requires separating area $S = |\vec{S}|$ and normal $\hat{n} = \vec{S}/S$, plus an auxiliary vector $\vec{A}$ built from vertex coordinate products:

$$\vec{\tau}_i = \frac{\rho g S_i}{12} \left[\left(12\bar{h} - 4 \sum y_k \right) \vec{x}_{\text{CoM}} + \vec{A} \right] \times \hat{n}_i$$

While a naive simulation computes torque by incorrectly applying the force at the geometric centroid, true hydrostatic pressure increases with depth. This creates an offset, a *Pressure Gradient Correction* (PGC), which shifts the true point of force downward to the Center of Pressure ($\vec{x}_{\text{CoP}}$). The bracketed expression in the equation above is a computationally efficient expansion of the true moment arm from $\vec{x}_{\text{CoP}}$ to $\vec{x}_{\text{CoM}}$. The auxiliary vector $\vec{A}$ computes the exact depth-weighted torque around the global origin, mathematically capturing the PGC organically. By combining this with the CoM term, the closed-form equation perfectly computes the true moment arm without ever calculating explicit CoP coordinates. This fundamentally eliminates the "ghost torques" caused by uniform centroid sampling.

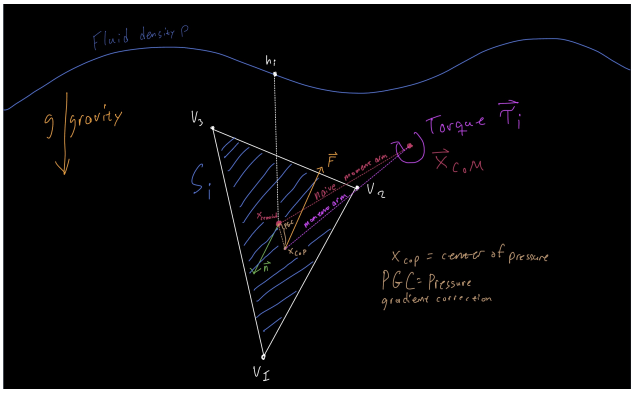

Figure 3: Per-triangle force and torque from hydrostatic pressure. The force acts through the center of pressure, offset from the centroid by the pressure gradient correction captured in the auxiliary vector.

3.3 The Clipping Problem

Triangles straddling the waterline must be clipped to isolate the submerged portion. Standard linear clipping interpolates between vertex positions using the signed distance to the water surface, producing a straight chord between the two edge intersection points. This is exact when the water surface is flat, but on curved Gerstner waves the true waterline follows the wave shape across the triangle face.

For triangles whose edges are short relative to the wavelength, the chord error is negligible. For coarser meshes in moderate waves, the regime targeted by this project, the error can produce incorrect submerged area estimates and corresponding force/torque errors.

4. Implementation

4.1 System Architecture

I designed the system to split computation between C# (Unity) and C++ (native DLL) across a P/Invoke boundary. The core architectural principle I followed is that the DLL is *wave-agnostic*: it receives per-vertex water heights and performs only geometric operations (transform, classify, clip, integrate). Wave evaluation lives entirely in C#, making the DLL reusable across wave models.

Per frame, the pipeline is:

1. **C#:** Transform mesh vertices to world space, sample Gerstner wave height at each vertex's (x, z) position.
2. **C# to C++ (P/Invoke):** Send the 4×4 transform matrix and per-vertex water heights array. Mesh geometry (vertices, indices) was sent once at creation.
3. **C++ (DLL):** Classify triangles via bitmask, clip intersecting triangles, accumulate force and torque via closed-form integrator. Return 6 floats.
4. **C# (Controller):** Apply force to Rigidbody, perform angular momentum damping.

Figure 4: Unity scene overview, Gerstner-wave plane with multiple `BuoyancyController` objects in the test scene.

4.2 Closed-Form Integrator

My C++ implementation mirrors Hirae's equations using GLM for vector math. I wrote one `accumulateBuoyancy` function that handles both fully submerged and clipped sub-triangles. The area vector is computed once via half-cross-product; the force equation uses it directly (no `sqrt`), while the torque

equation extracts area and normal separately (one `sqrt` per triangle).

Algorithm: Linear closed-form buoyancy path

```
Input: world vertices V, triangle indices
    I, water heights H
Output: total force F and torque tau

F, tau <- 0
for each triangle (a, b, c) in I:
  depth <- H - V.y at a, b, c
  mask <- submerged-bitmask(depth)
  if mask == dry:
    continue
  if mask == fullySubmerged:
    accumulateBuoyancy(a, b, c, H)
  else:
    clipped <- linearClip(a, b, c, H, mask)
    for each subTriangle in clipped:
      accumulateBuoyancy(subTriangle)
return F, tau
```

During implementation, I spent hours debugging a persistent residual torque only to discover a typographical error in the paper's Eqs. 7–9. The auxiliary vector components A_x and A_z are printed with a factor $\delta = 2\sum y_i - 4h$, but re-deriving the integral from the product-of-linears identity myself showed the correct factor is $\delta_{xz} = \sum y_i - 4h$. Only A_y uses the doubled factor, due to the algebraic identity $(\sum y_i)^2 = \sum y_i^2 + 2\sum_{i<j} y_i y_j$. This error produces a persistent residual torque that prevents correct equilibrium (§5.1).

4.3 Triangle Classification and Linear Clipping

Each triangle is classified by a 3-bit bitmask encoding which vertices are submerged (vertex a = bit 2, b = bit 1, c = bit 0). A small epsilon (10^{-4} m) prevents degenerate zero-area triangles when vertices sit exactly on the waterline.

The bitmask dispatches to one of two clipping functions via `switch`. One-below, the submerged region is a single triangle (submerged vertex + two edge intersection points). Two-below, the submerged region is a quadrilateral split into two triangles. Crucially, vertices are passed in *cyclic permutation* order, not swapped, to preserve winding and keep normals pointing outward.

Intersection points are found by linear interpolation along edges using signed distance to the water surface:

$$t = \frac{d_{\text{above}}}{d_{\text{above}} - d_{\text{below}}}, \quad \vec{p} = \text{lerp}(\vec{v}_{\text{above}}, \vec{v}_{\text{below}}, t)$$

The water height at the clip point is interpolated with the same parameter t.

4.4 Wave Sampling

The test scenes use Catlike Coding's Gerstner-wave implementation (Flick, 2018), but the wave model is not the core contribution. The important requirement is that physics and rendering sample the same surface. The URP shader and the C# buoyancy sampler therefore share the same wave parameters.

Because Gerstner waves horizontally displace vertices, buoyancy needs an inverse query: given world (x, z), find the rest position whose displaced (x, z) matches. For the moderate steepness values used in the tests, two fixed-point iterations were enough:

$$\vec{p}_0^{(n+1)} = (x, z) - \Delta_{xz}(\vec{p}_0^{(n)})$$

4.5 Angular Momentum Damping

Unity's built-in `angularDrag` scales angular velocity $\vec{\omega}$ directly, which removes different amounts of angular momentum on different axes depending on the inertia tensor. Following Hirae, the implementation damps angular momentum $\vec{L}$ instead:

$$\vec{L}_{\text{damped}} = \alpha \cdot \vec{L}, \quad \vec{\omega} = \mathbf{I}^{-1}\vec{L}_{\text{damped}}$$

The inertia tensor $\mathbf{I}$ is diagonal only in its principal-axis frame, so the implementation rotates $\vec{L}$ into that frame via the tensor rotation quaternion, divides componentwise, and rotates back. A coefficient of $\alpha = 1$ means no damping and can leave objects spinning indefinitely, while $\alpha = 0$ removes angular momentum in one step. Lower values therefore damp more aggressively; the chosen value preserves wave-driven bobbing while quickly killing parasitic spin.

4.6 Adaptive Curved Clipping

The core contribution. For each triangle classified as intersecting the waterline, the linear clipper produces two clip points P_1 and P_2. The adaptive algorithm refines the depth information along the chord between them. This is an extension beyond Hirae's paper, which uses the straight linear clip produced by the triangle-waterline intersections.

Algorithm: Adaptive Curved Clipping

```
Input: clip points P1, P2; submerged
    vertex(s); refineSamples N
Output: fan-triangulated sub-mesh with
    corrected water heights

polyVerts    <- [P1]
polyHeights  <- [h(P1)]
for k = 1 to N:
  u  <- k / (N + 1)
  m  <- lerp(P1, P2, u)
  hm <- WaveManager.SampleHeight(m.x, m.z,
      t)
  polyVerts.append(m)
  polyHeights.append(hm)
polyVerts.append(P2)
polyHeights.append(h(P2))
Fan-triangulate from submerged vertex
    through polyVerts
Pass sub-triangles + heights to DLL via
    ComputeBuoyancyFromTriangles
```

The key insight is that this is *integration refinement*, not *geometric refinement*. The sub-triangle vertices remain on the original triangle plane, the geometry stays flat. The correction enters through the per-vertex water

heights, which now follow the true wave surface instead of being linearly interpolated from the endpoints.

When a chord sample sits below a wave crest ($h_m > m_y$), that sub-triangle receives extra positive depth, crediting the buoyancy from water bulging into what the linear chord classified as dry. When a sample sits above a trough ($h_m < m_y$), depth goes negative, debiting spurious buoyancy from a region that is actually dry. The closed-form integrator handles negative depths naturally, no special-case polygon cutting is required.

An early version only inserted refined vertices at sign-change crossings, which corrected for crests but silently ignored troughs (no sign change occurs when the wave dips below the chord without crossing it). The fix was to sample unconditionally at every chord point, making the correction symmetric.

The DLL exposes a second entry point, `ComputeBuoyancyFromTriangles`, that accepts pre-clipped sub-triangles with per-vertex water heights and accumulates force/torque using the same `accumulateBuoyancy` function as the linear path. No code is duplicated.

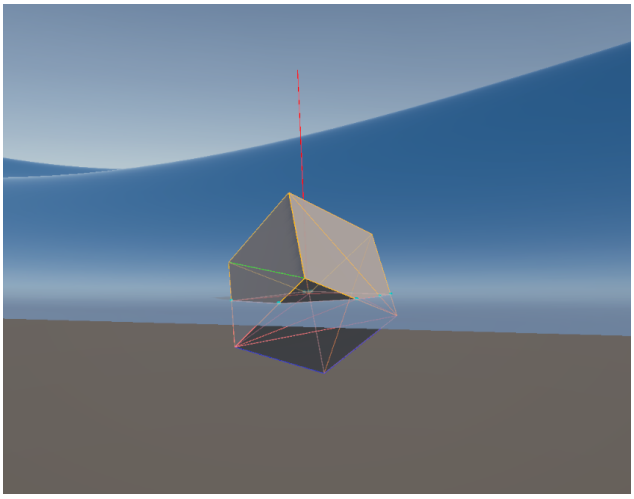

Figure 5: Linear clipping gizmo. Green = dry, blue = fully submerged, orange = intersecting.

4.7 Performance Optimizations

Runtime mesh simplification. The Stanford bunny (30k triangles) initially produced 82 ms FixedUpdate times, but that measurement was taken before Burst-compiled height sampling. Using Garland-Heckbert (1997) quadric edge-collapse (via UnityMeshSimplifier), the final buoyancy mesh was reduced to roughly 7k triangles at startup. The visual mesh is unaffected.

Burst-compiled wave sampling. Per-vertex Gerstner evaluation was the dominant cost (76% of FixedUpdate). Moving the sampling loop to a `BurstCompile IJobParallelFor` using `Unity.Mathematics` intrinsics yielded a significant speedup, I would estimate it at around 6-10x, through SIMD vectorization and multi-core dispatch.

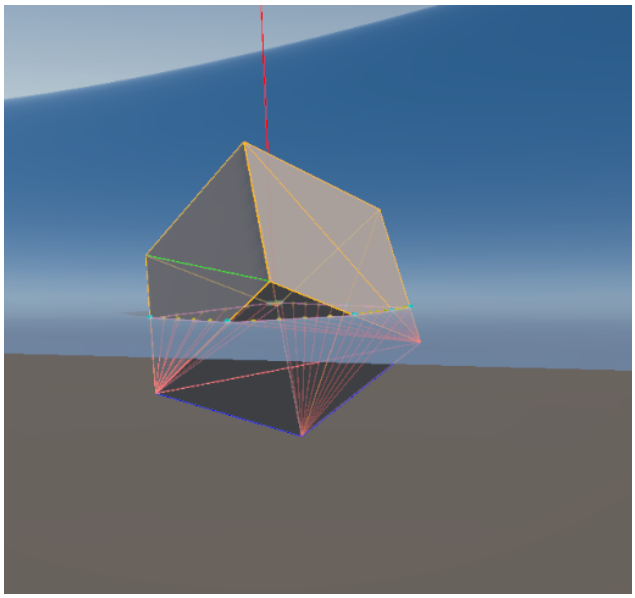

Figure 6: Adaptive clipping gizmo. The magenta line and yellow dots show refined waterline samples along the chord.

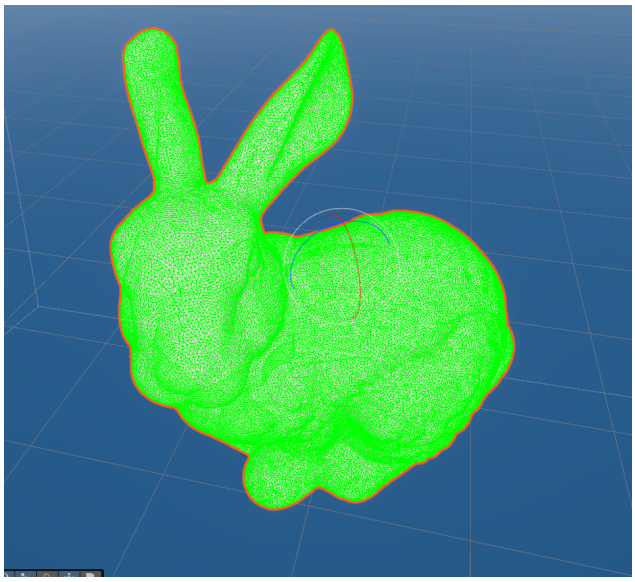

Figure 7: Bunny before simplification, 30,000 triangles used as ground-truth in §5.2.

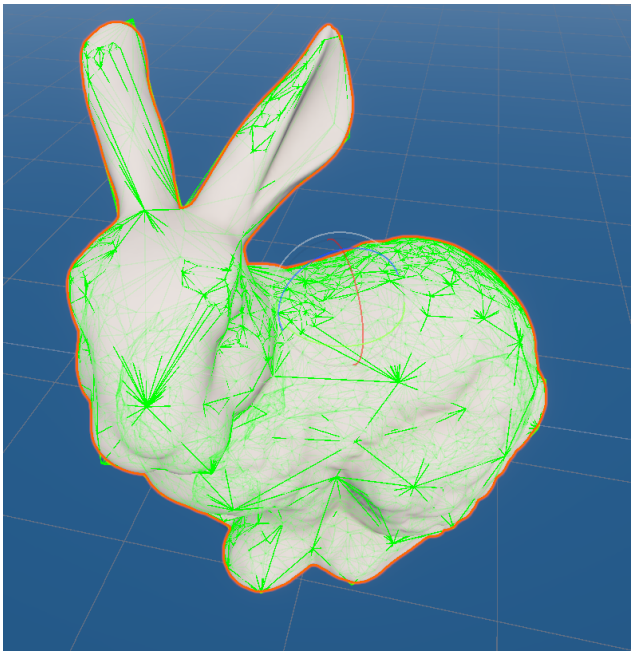

Figure 8: Bunny after Garland-Heckbert decimation to roughly 7k triangles.

5. Results

5.1 Accuracy Validation

Force sweep. A 1×1×1 cube swept through flat water in 25 steps from fully above to fully below the surface. Computed buoyancy force matches the analytical $F_b = \rho g V_{\text{sub}}$ to within 10^{-6}%, floating-point noise. The closed-form integrator is exact on flat water.

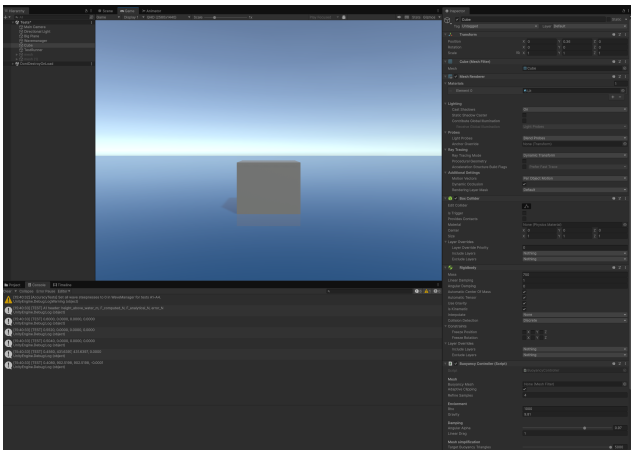

Figure 9: Force sweep test scene.

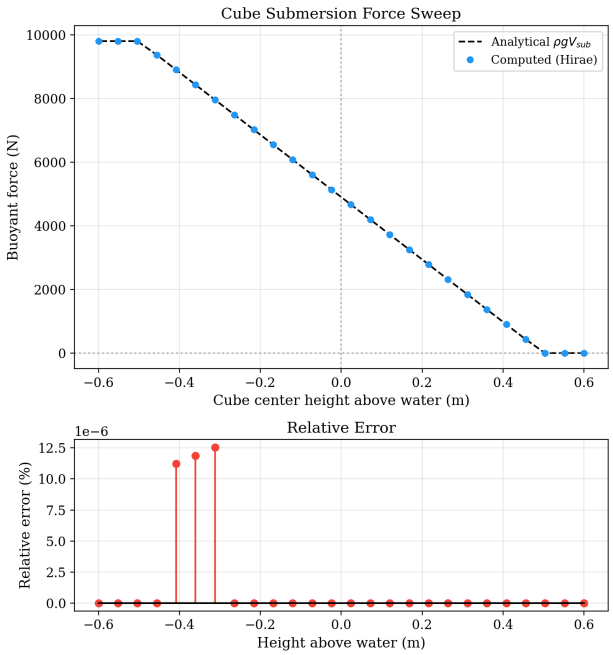

Figure 10: Force sweep. Computed and analytical force overlap; relative error stays near machine precision.

Equilibrium tilt. A 1 m³ cube with density ratio 0.75, constrained to one rotation axis so that its cross-sectional equilibrium matches the long-rectangular-bar case analyzed by Igarashi & Nakamura (2007). The analytical equilibrium tilt for that one-axis case is $\arctan(0.5) = 26.565$ř.

With our corrected formula: settles to **26.565°**, residual torque $\mathbf{2.4 \times 10^{-5}}$ N m.

With the paper's printed Eqs. 7–9: settles to **45.000°**, permanent residual torque **215.9** N m. The force equation (Eq. 5) is correct in both versions, only the torque auxiliary vector carries the error.

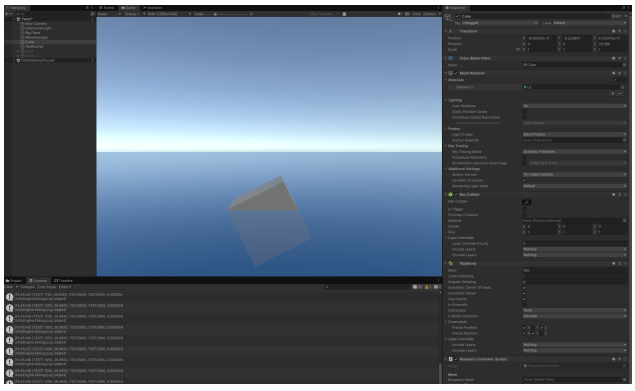

Figure 11: Corrected formula settling at 26.565°.

This constitutes empirical evidence of a typographical error in Hirae et al. (2025). The factor-of-2 discrepancy in A_x and A_z was independently confirmed by re-deriving the surface integral from the product-of-linears identity (see the derivation notes in the blog post). I'm going to contact the authors.

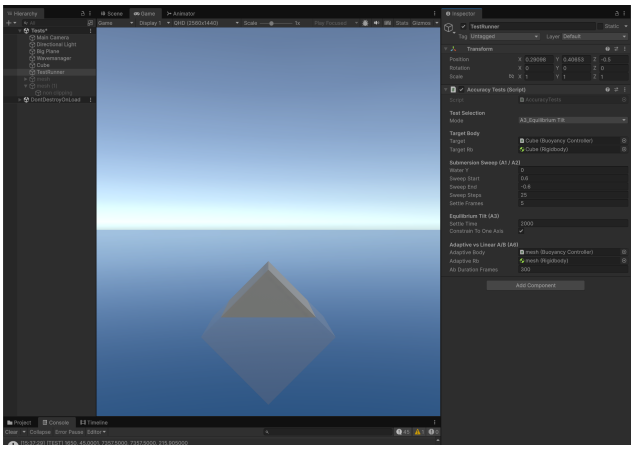

Figure 12: Printed formula settling at 45°.

quilibrium Tilt - 1 m^3 cube, density ratio = 0.75, Corrected vs Paper Formu

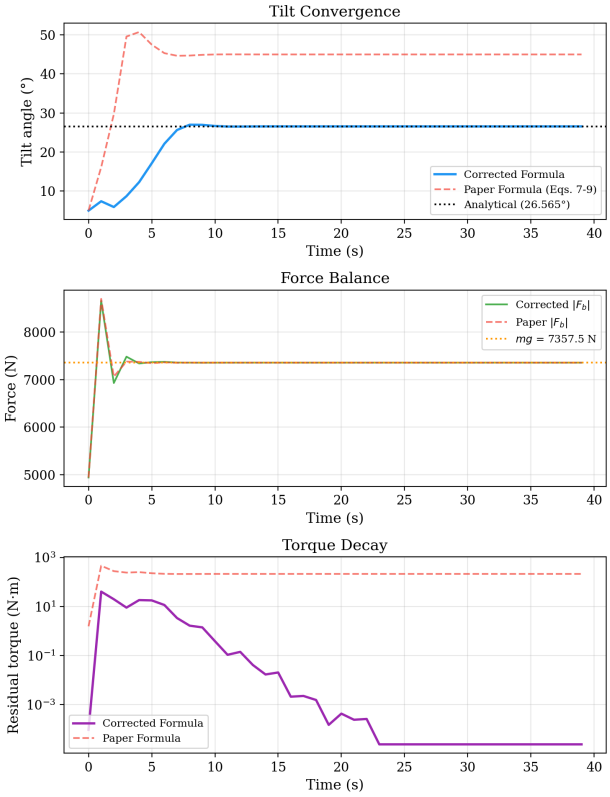

Figure 13: Equilibrium tilt comparison. The corrected formula reaches 26.565° with residual torque near zero; the printed formula sticks at 45°.

5.2 Adaptive vs Linear Clipping

To evaluate the adaptive algorithm against a meaningful baseline, I use a *dense mesh convergence argument*. A 30,000-triangle bunny processed through the linear clipper serves as ground truth, its triangles are so small relative to the wavelength that the chord should perfectly approximate the local waterline.

Three roughly 7k-triangle bunnies are simulated alongside the 30k ground truth, each dropped into identical Gerstner waves:

1. **Linear Path** (standard clipping)
2. **Adaptive N=4** (4 refinement samples per chord)
3. **Adaptive N=16** (16 samples per chord)

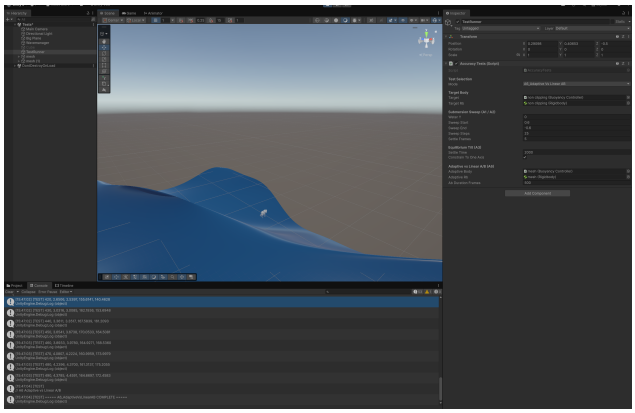

Figure 14: Adaptive clipping comparison scene, captured late in the run after the objects had drifted from their initial spacing.

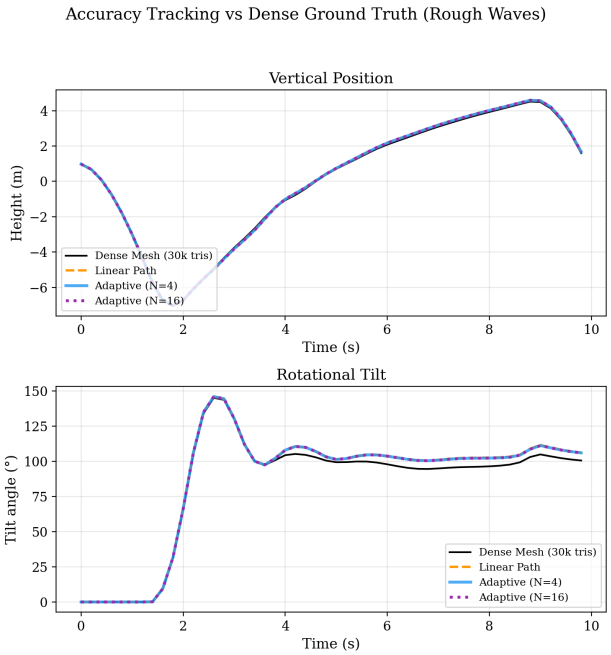

Figure 15: Trajectory tracking vs dense-mesh ground truth. The methods stay close in height; tilt differs by about 6° in rough waves.

Three key findings:

1. **N=4 is fully converged.** The N=4 and N=16

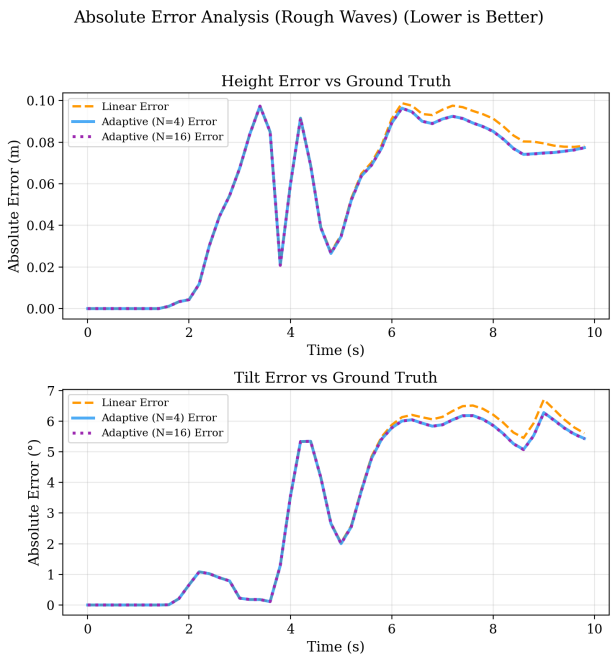

Figure 16: Absolute error analysis. N=4 and N=16 overlap, and the linear-adaptive gap is small compared with the roughly 6° tilt difference.

error curves are identical, proving that 4 refinement samples exhaust the algorithm's correction capacity for this mesh density.

2. **Residual mesh and dynamics error dominates.** The remaining ~6° tilt gap is not caused by a visibly broken low-poly bunny; the 7k mesh still preserves the main shape. It is better explained by remaining mesh-volume differences and the sensitivity of floating motion in rough waves. This error cannot be removed by clipping refinement.
3. **Adaptive vs linear improvement is negligible.** The gap between the linear (orange) and adaptive (blue) error curves is less than 0.5°, completely swamped by the 6° volume error.

5.3 Performance

To push the system to its limits, I set up a stress test with 100 spheres on Gerstner waves and measured performance using a custom 1-second-window FPS logger. All measurements were recorded on an AMD Ryzen 9 5900X CPU and AMD Radeon RX 7900 XTX GPU:

Configuration	Avg FPS	Steady-State FPS
Linear (C++ DLL)	363	300–415
Adaptive N=1 (C#)	53	3
Adaptive N=2 (C#)	10	2

The adaptive path degrades over time because at equilibrium, a maximal number of triangles straddle the waterline permanently. Each intersecting triangle triggers `SampleHeight` calls (Newton iterations with trigonometric evaluations) through managed C#, plus `List<T>` allocations that pressure the garbage collector.

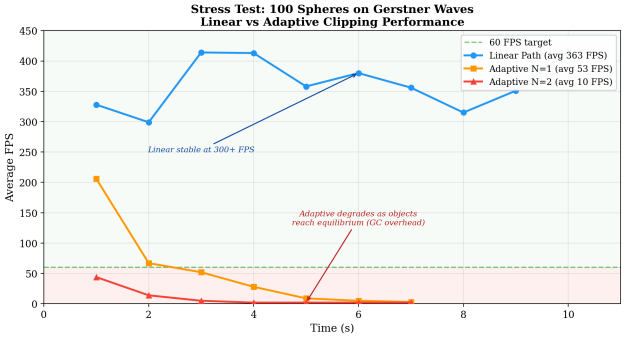

Figure 17: Stress test: 100 spheres. Linear path sustains 363 FPS average. Adaptive paths degrade as objects reach equilibrium and more triangles straddle the waterline, triggering managed-code GC pressure.

5.4 Visual Results

Beyond the quantitative tests above, the system was exercised in a scene designed to demonstrate stability across object types, mesh complexity, and concurrent counts. The sphere scene below corresponds to the performance stress test in §5.3.

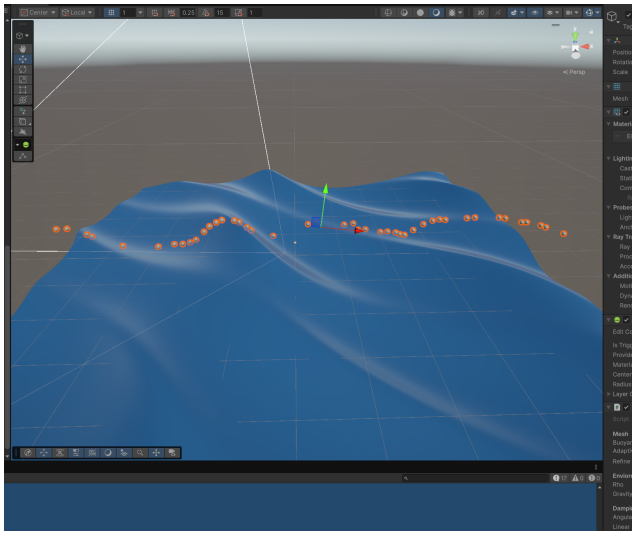

Figure 18: 100-sphere stress scene using the linear C++ path.

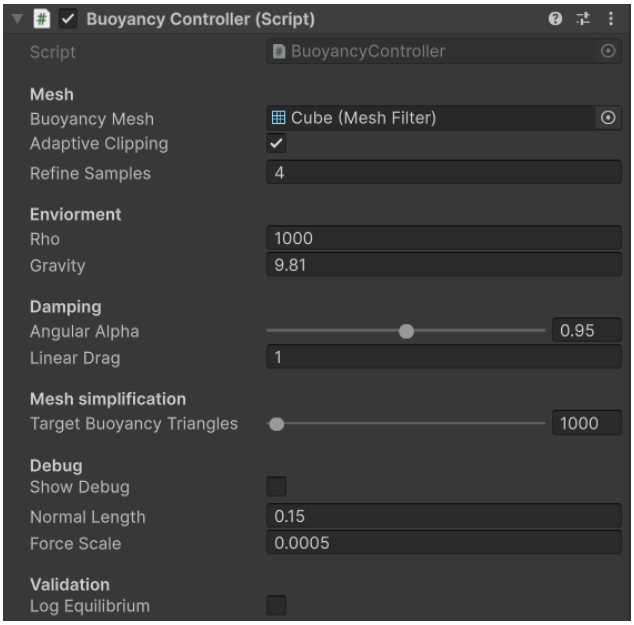

Figure 19: BuoyancyController settings used in the experiments.

6. Evaluation

6.1 Evaluation Methods

Buoyancy simulation evaluation in the literature follows two approaches. Hirae et al. validate against analytical equilibrium angles derived from the long-rectangular-bar result of Igarashi & Nakamura (2007), confirming numerical convergence of force and torque. Fábián reports wall-clock timing benchmarks across mesh resolutions. Neither conducts perceptual evaluation, whether mathematical accuracy differences produce visible differences in simulation quality remains unstudied.

6.2 Proposed Perceptual Study

The numerical results in §5.2 show that the adaptive and linear paths are almost the same on my test scene. The more useful question for a game is whether a player would notice the difference. If I had more time, I would test that directly with a simple boat hull rather than the bunny, because people have a clearer intuition for how boats should float.

Hypothesis 1. Observers cannot reliably distinguish linear clipping from adaptive clipping on simplified meshes in moderate waves.

Hypothesis 2. Observers may prefer the dense-mesh reference over the simplified versions in rough waves, but the simplified motion will look only slightly off rather than obviously wrong. A 6° tilt difference is measurable in plots, but it is probably not enough for most non-experts to confidently call the motion incorrect.

Design. I would run a within-subjects two-alternative forced choice (2AFC) study. Each trial would show two synchronized 10-second clips of the same boat floating on the same wave field, and the participant would choose which looks more physically realistic.

Stimuli. The clips would compare three conditions: *Linear*, *Adaptive N=4*, and *Dense Reference*. I would record calm and rough waves, keep camera/lighting/shader settings fixed, and change only the buoyancy code path.

Participants and metrics. Around 12 KTH students is enough for a small course-project perceptual study following the Hwang & Salvendy (2010) `10 ± 2` rule. I would measure which clip they choose, response time, confidence, and a short free-text explanation of what cue they used.

This study has not been conducted. It is included as a practical next step for connecting numerical buoyancy error to visible simulation quality.

6.3 Discussion

The adaptive algorithm assumes at most one waterline crossing per triangle edge, valid only when edge length is less than the smallest wavelength in the wave field. Gerstner waves with steepness approaching 1.0 produce cusps that violate this assumption. The mesh simplifier occasionally produces degenerate slivers that trigger the epsilon guard. The buoyancy mesh does not have to be perfectly watertight for the code to run, but normals must be consistently outward-facing, holes and non-manifold regions change the physical meaning of the displaced volume.

While numerous minor bugs were encountered during development, only the most instructive failure modes are detailed here. The first was a sub-triangle winding-order bug, sub-triangles passed in swap order rather than cyclic permutation produce inverted normals on the two-below case, so the buoyancy force flips sign on half the clipped triangles and the body sinks erratically. The second was the symptom of the printed Eq. 7–9 typo discussed in §5.1: when locked to one axis, the cube settles at 45° with a permanent residual torque of 215.9 N m instead of the analytical 26.565°, and when unconstrained, it spins indefinitely. However I met bugs all the time along the way and only recorded a handful in the blog posts I wrote.

In practice, linear clipping is sufficient for real-time game use in this project. Its error is already so small that I would not have mistaken it for anything other than real buoyancy. Unfortunately, the adaptive path adds large overhead for a minuscule visual gain. Consequently, the proposed adaptive clipping implementation does not justify its performance overhead for real-time constraints.

The adaptive algorithm does not geometrically trace the wave-triangle intersection curve. My original plan was to do that, but it quickly became clear that solving a nonlinear surface intersection robustly would be a separate project. Instead, the implemented version corrects the depth values at sample points along the straight chord, letting the closed-form integrator naturally account for the curved waterline through its linear pressure interpolation. This is much simpler and reuses

the existing integrator without modification.

7. Conclusion and Future Work

This project implemented Hirae's closed-form surface-pressure integrator as a Unity-native C++ plugin, introduced Adaptive Curved Clipping as a waterline refinement scheme, and empirically evaluated both against analytical baselines and dense-mesh ground truth.

Key findings:

1. The integrator reproduces analytical solutions to machine precision on flat water and matches the 26.565° equilibrium tilt within 10^{-5} N m residual torque. A typographical error in the paper's printed equations was identified and corrected.
2. Adaptive Clipping with N=4 samples is mathematically converged, but provides negligible accuracy improvement ($< 0.5°$) over linear clipping in the tested scenes. As implemented here, adaptive clipping is probably not worth shipping for real-time use.
3. The linear C++ path sustains 300+ FPS with 100 simultaneous rigid bodies. The adaptive C# path is unsuitable for real-time use in its current form.

Future work includes implementing visual two-way fluid-rigid body coupling, allowing objects to locally deform the wave surface via interactive heightmaps or ripple shaders. Additionally, future efforts will focus on executing the boat-based perceptual study to quantify the visual impact of mathematical accuracy.

8. Acknowledgments

Christopher Peters and the DH2323 course team at KTH. Jasper Flick (Catlike Coding) for the MIT-licensed Gerstner wave implementation.

Artificial intelligence tools (Gemini 3.1 pro through https://gemini.google.com/app) were used during this project for proofreading and refining the written text of the blog post and this report. During development, AI was utilized as a query engine for documentation lookups (such as checking for specific functions in the GLM library) and to help understand the setup for the C# to C++ `P/Invoke` boundary. Outside of these scopes, no AI was used during the project.

References

- Fábián, G. (2025). *Approximate and exact buoyancy calculation for real-time floating simulation of meshes.* In *EUROGRAPHICS 2025*, D. Ceylan and T.-M. Li (Eds.), Short Papers. https://doi.org/10.2312/egs.20251031
- Flick, J. (2018). *Waves: Moving Vertices, Catlike Coding.* Published July 25, 2018. https://catlikecoding.com/unity/tutorials/flow/waves/
- Garland, M. & Heckbert, P. (1997). *Surface simplification using quadric error metrics.* SIGGRAPH '97.
- Hirae, H., Morishima, S., & Ando, R. (2025). *An Analytical Integrator for Solid-Fluid Coupled Buoyancy Forces.* In *SIGGRAPH Asia 2025 Technical Communications* (SA Technical Communications '25), December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3757376.3771383
- Hori, T. (2021). *Proof that the Center of Buoyancy is Equal to the Center of Pressure by means of the Surface Integral of Hydrostatic Pressure Acting on the Inclined Ship.* Research note, Nagasaki Institute of Applied Science Bulletin, 61(2), 135–154.
- Hwang, W. & Salvendy, G. (2010). *Number of people required for usability evaluation: the `10 ± 2` rule.* Communications of the ACM 53(5), 130–133. https://doi.org/10.1145/1735223.1735255
- Igarashi, T. & Nakamura, H. (2007). *An Investigation on the Orientation of a Long Rectangular Bar in Still Water.* Journal of Japan Society of Fluid Mechanics 26(6), 393–400. https://doi.org/10.11426/nagare1982.26.393

Appendix A: Project Specification

The original project proposal was submitted separately. The code and generated project materials are available in the GitHub repository linked above, GitHub Repo

Attached below is the project proposal.

DH2323 Project Specification

Felix Stenberg | felixste@kth.se | 0210314274

Target Grade: A–B

crispigt.com/dh2323 | github.com/Crispigt/DH2323-WaveBuoyancy

---

The Idea

I want to build a real-time buoyancy simulation in Unity where meshes representing rigid hulls (boats, bunnies, etc.) float realistically on dynamic, wavy water. The simulation targets water-tight meshes with consistently outward-oriented normals, i.e. the kind you'd use as a collision hull, not an art asset with open edges or double-sided faces (though it might inadvertently work for other meshes as well). The goal is not just "push the center of mass up when it's below the surface", but to actually compute the correct forces and torques per triangle so that objects tilt, rock, and settle naturally. I got interested in this because in the Game Design course our game is very much depending on buoyancy simulation and we felt the built-in Unity solution was not so good.

The main inspiration comes from three papers,

- Fábián (2025) wrote a Eurographics short paper on buoyancy for arbitrary meshes. His approach uses volumes (tetrahedra) which works great for flat water, but gets quite messy when you try to extend it to wavy surfaces since you would need to compute curved "caps" where the water cuts through the mesh.
- Hori (2021) proved that you can actually skip volumes entirely, just integrate hydrostatic pressure over the submerged *surface* of the mesh and you get the exact same result. No caps needed.
- Hirae et al. (2025) from SIGGRAPH Asia took this further. The standard way to do surface pressure integration is to sample depth at each triangle's centroid, but this introduces so called "ghost torques" on coarse meshes (the torque integral is not polynomial, so centroid sampling cannot capture it exactly). Hirae derived a closed-form solution that computes the force and torque per triangle exactly using just the vertex coordinates, without any sampling error.

My plan is to implement Hirae's closed-form integrator in a C++ DLL that Unity calls every physics frame. The heavy math runs in C++, and Unity just passes in the mesh transform and wave parameters and gets back a force + torque vector.

Hirae's paper handles dynamic waves by clipping triangles against the water surface using lerp between the wave heights at the vertices. This works well when the triangles are small relative to the wavelength, but for larger triangles it assumes the wave is locally flat, which it is not.

My main contribution would be something I call Adaptive Curved Clipping. Instead of making a single straight cut across a partially submerged triangle, the idea is to sample N evenly-spaced points along the line between the two edge intersections. By evaluating the actual wave height at each sample and checking for sign changes, we find a refined set of crossing positions directly on the triangle plane.

This creates a piecewise-linear waterline that closely follows the wave's actual curvature. We then fan-triangulate from the submerged vertex to each segment of this new polyline, yielding much more accurate submerged sub-triangles.

The method assumes at most one waterline crossing per edge, which is valid as long as the triangle edges are smaller than the shortest wavelength. Thus, it primarily targets the coarse-mesh / moderate-wave regime. As a stretch goal, the fixed-N sampling could be upgraded to recursive bisection. This would automatically adapt the number of samples to the local curvature, though it is more computationally costly and harder to implement.

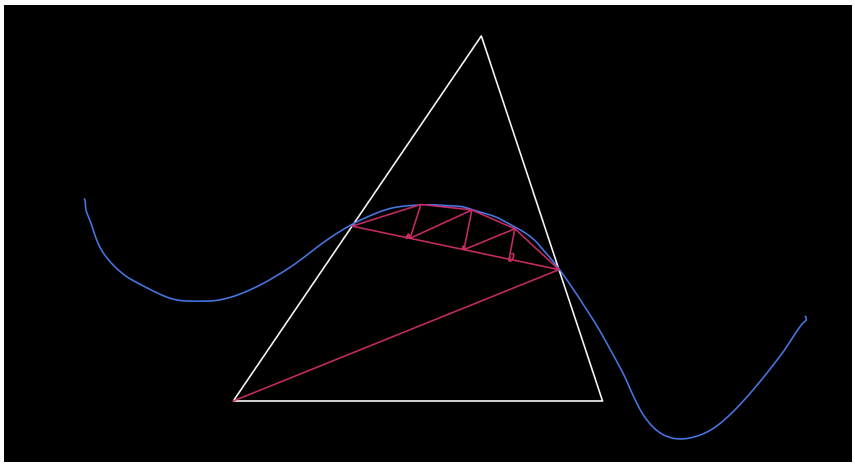

Figure 1: Adaptive curved clipping illustration

So concretely, I'll be implementing Hirae's closed-form integrator then I'll implement and compare three clipping approaches:

1. Flat water baseline, everything clips against y=0, the simplest case
2. Linear wave clipping, Hirae's approach, lerp between vertex wave heights
3. Adaptive curved clipping, my extension, fixed-N waterline sampling (with recursive bisection as a stretch goal)

Architecture

- C++ DLL handles all the buoyancy computation (vertex transforms, triangle classification, clipping, force/torque integration). Built with CMake, loaded by Unity via P/Invoke.
- Unity (C#) manages the scene, sends mesh data to the DLL once at startup, then each frame sends the current transform matrix + wave parameters and applies the returned forces to the Rigidbody.
- The wave function (sum of sines) runs in both the C++ DLL (for physics) and a Unity vertex shader (for visuals), using the same parameters so they stay in sync.

How the simulation will work

Every physics frame (50 Hz), the C++ DLL will run a pipeline like this for each floating object:

1. Unity sends the object's 4×4 transform matrix. The DLL applies it to every vertex of the mesh (same as model transforms in a raytracer).
2. For each triangle, check if its three vertices are above or below the water surface,
    - All below -> fully submerged
    - All above -> skip it
    - Mixed -> partially submerged, needs clipping

3. Find where the triangle's edges cross the water surface and cut the triangle along that line. This produces 1–2 smaller sub-triangles that are fully submerged. (This is where the three clipping methods differ, see above.)

4. Compute force and torque per submerged triangle. This is where Hirae's closed-form integrator comes in. For a triangle with vertices at positions (x_1, y_1, z_1), (x_2, y_2, z_2), (x_3, y_3, z_3) and wave surface height h_i at the centroid:

   Force per submerged triangle (Eq.,5)::

$$\vec{F}_i = \frac{\rho g S_i}{3}(-3h_i + y_1 + y_2 + y_3)\,\vec{n}_i$$

Torque per triangle (about the center of mass):

$$\vec{\tau}_i = \frac{\rho g S_i}{12}\left[\{12h_i - 4(y_1 + y_2 + y_3)\}\vec{x}_{\text{CoM}} + \vec{A}\right] \times \vec{n}_i$$

Where: - ρ = fluid density, g = gravity - S_i = area of the triangle - $\vec{n}_i$ = outward unit normal of the triangle - $\vec{x}_{\text{CoM}}$ = center of mass of the rigid body - $\vec{A}$ = auxiliary vector built from the vertex coordinates (three components, each just additions and multiplications, no sine, sqrt, or anything expensive). The full expansion is in Hirae's paper (Equations 5–9).

5. Sum all forces and torques. Return 6 floats to Unity (3 for force, 3 for torque).

6. Apply angular momentum damping. Hirae showed that damping angular velocity directly (which is what Unity's built-in angularDrag does) produces artifacts. Instead, we should damp angular momentum $\vec{L}$ by scaling it each frame: $\vec{L}_{\text{damped}} = \alpha \cdot \vec{L}$, then derive angular velocity from that.

Here are some pictures for more clarity on the calculations:

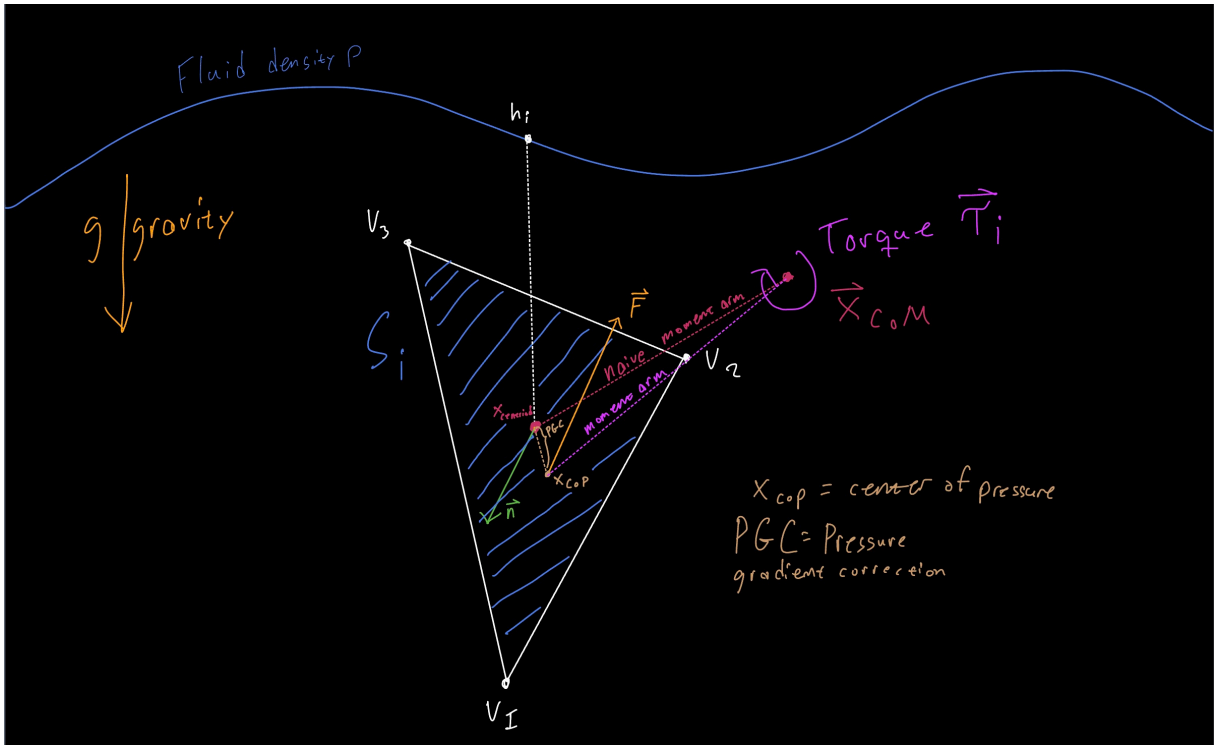

Figure 2: Per-triangle force $\vec{F}_i$ and torque $\vec{\tau}_i$ from hydrostatic pressure.

Force per submerged triangle (Eq. 5):

$$\vec{F}_i = \frac{\rho\, g\, S_i}{3}\,(-3h_i + y_1 + y_2 + y_3)\,\vec{n}_i$$

Torque per triangle about CoM (Eq. 6):

$$\vec{\tau}_i = \frac{\rho\, g\, S_i}{12}\left[\underbrace{\{12h_i - 4(y_1{+}y_2{+}y_3)\}\,\vec{x}_{\mathrm{CoM}}}_{\text{CoM term}} + \underbrace{\vec{A}}_{\text{correction}}\right] \times \vec{n}_i$$

Auxiliary vector $\vec{A} = (A_1,\, A_2,\, A_3)^\top$ (Eqs. 7–9), letting $\delta = -4h_i + 2(y_1{+}y_2{+}y_3)$:

$$\begin{aligned} A_1 &= (x_1{+}x_2{+}x_3)\,\delta + x_1y_1 + x_2y_2 + x_3y_3 \\ A_2 &= (y_1{+}y_2{+}y_3)\,\delta - 2(y_1y_2 + y_2y_3 + y_3y_1) \\ A_3 &= (z_1{+}z_2{+}z_3)\,\delta + z_1y_1 + z_2y_2 + z_3y_3 \end{aligned}$$

While a naive simulation computes torque by incorrectly applying the force at the geometric centroid (the red naive moment arm), true hydrostatic pressure increases with depth. This creates a offset, a Pressure Gradient Correction (PGC), which shifts the true point of force downward to the Center of Pressure ($\vec{x}_{\mathrm{CoP}}$). The bracketed expression in Equation 6 is a computationally efficient expansion of the true moment arm ($\vec{x}_{\mathrm{CoP}}, \vec{x}_{\mathrm{CoM}}$). The auxiliary vector $\vec{A}$ computes the exact depth-weighted torque around the global origin, mathematically capturing the PGC organically. By combining this with the CoM term (which shifts the rotation pivot to the object's center of mass), Hirae's equation perfectly computes the true moment arm without ever calculating explicit CoP coordinates, very neat.

How it could be evaluated

- Compare computed buoyancy forces against analytical solutions for simple shapes (e.g. a cube) at various submersion depths
- Benchmark the C++ DLL across meshes of different sizes, roughly ~1k to ~100k triangles, and maybe also look at multithreading if I have time for that
- Side-by-side visual comparisons of objects floating in wavy water with linear vs. adaptive clipping. Also comparing centroid-sampling (the naive approach) against Hirae's closed-form integrator to show the "ghost torque" problem on coarse meshes
- Write up a proposed perceptual study where participants compare linear vs. adaptive clipping in stormy water conditions to see if people can actually tell the difference visually (I think this is required for the grade?)

Questions for you guys

- Is the math explained well enough in this proposal for me to just put in the final report? Is there something you did not understand that I should explain more clearly?
- Is three sources enough?
- Is the proposed user study needed? How indepth does that have to be?
- Is comparing three clipping algorithms (flat baseline, linear, adaptive) enough scope for an A–B grade or should I aim for more?
- Any other tips on the user study proposal section?
- Would it also be useful to compare against Fábián's volume-based approach as a baseline, or is that overkill? It only works for flat water surfaces but his code is available on GitHub so it would not be too much extra work.

- Is the overall scope appropriate for an A–B grade, or would you recommend changing anything?

References

- Fábián, G. (2025). *Approximate and exact buoyancy calculation for real-time floating simulation of meshes.* Eurographics 2025 Short Paper.
- Hirae, H., Morishima, S., & Ando, R. (2025). *An Analytical Integrator for Solid-Fluid Coupled Buoyancy Forces.* SIGGRAPH Asia 2025 Technical Communications.
- Hori, T. (2021). *Proof that the Center of Buoyancy is Equal to the Center of Pressure by means of the Surface Integral of Hydrostatic Pressure Acting on the Inclined Ship.* Nagasaki Institute of Applied Science.